

30 DBMS + SQL QUESTIONS + 20 Project Specific Question

1) What is DBMS?

They want to see whether you understand **why databases are used instead of files**.

2) Difference between DBMS and RDBMS

Why:

This is a very standard fresher question.

They check whether you know:

- normal database
 - relational database
 - table-based storage
 - relationships
-

3) What is Primary Key?

Why:

To test understanding of **uniqueness and entity identification**.

Expected:

- unique
 - not null
 - identifies each row
-

4) What is Foreign Key?

Why:

They want to know whether you understand **table relationships**.

This is extremely important for backend / SDE role.

Example:

`Orders(user_id) REFERENCES Users(id)`

5) Difference between Primary Key and Unique Key

Why:

They test depth.

Freshers often confuse this.

Expected:

- PK → unique + not null
- unique → null allowed

6) Can we create foreign key in same table?

Why:

Tests real schema design thinking.

Very commonly asked.

Example:

Employee(id, manager_id)

manager_id references Employee(id)

This is self-referencing relation.

7) What is normalization?

Why:

To check database design skills.

Interviewer wants to see if you can **avoid redundancy**, anomalies.

8) Explain 1NF, 2NF, 3NF, BCNF

Why:

One of the most asked DBMS questions.

This tests schema design and anomalies.

9) What is denormalization?

Why:

Checks whether you understand **tradeoff between performance and redundancy**.

This is often asked as follow-up.

10) What are ACID properties?

Why:

This is probably **top 3 most asked DBMS question**.

Expected:

- Atomicity
 - Consistency
 - Isolation
 - Durability
-

11) What is transaction?

Why:

To see whether you understand multiple queries as a single unit.

Example:

money transfer

debit + credit

12) What is COMMIT and ROLLBACK?

Why:

Checks practical SQL transaction knowledge.

13) What is dirty read?

Why:

This is concurrency control question.

Common in product companies.

14) What is deadlock?

Why:

Tests real-world system understanding.

Expected:

two transactions waiting for each other forever.

15) What is indexing?

Why:

Very important for SDE.

Interviewer wants to test **performance knowledge**.

16) Why does index improve performance?

Why:

Tests internal understanding.

Expected:

faster search using B-tree / index structure

17) Why insert/update is slower with index?

Why:

This is a favorite follow-up.

Expected:

because index structure must also be updated.

18) Difference between clustered and non-clustered index

Why:

Tests deeper DB understanding.

19) Difference between DELETE, DROP, TRUNCATE

Why:

Almost guaranteed SQL question.

20) Difference between WHERE and HAVING

Why:

Tests SQL execution flow.

Very common.

21) Difference between INNER JOIN and LEFT JOIN

Why:

Joins are asked in almost every interview.

22) Types of joins

Why:

Tests SQL basics.

Expected:

- inner
 - left
 - right
 - full
 - self join
-

23) Write query for second highest salary

Why:

This is the **most asked SQL coding question**.

```
SELECT MAX(salary)
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

24) Write query for Nth highest salary

Why:

Tests subquery + ranking logic.

25) Find duplicate rows

Why:

Tests GROUP BY + HAVING

```
SELECT email, COUNT(*)  
FROM Users  
GROUP BY email  
HAVING COUNT(*) > 1;
```

26) Top 3 highest salaries department wise

Why:

Very strong SQL interview question.

Tests window functions.

```
SELECT *  
FROM (  
    SELECT *,  
        DENSE_RANK() OVER(PARTITION BY dept_id ORDER BY salary DESC) rnk  
    FROM Employee  
) t  
WHERE rnk <= 3;
```

27) Find employees without department

Why:

Tests LEFT JOIN + NULL logic

28) Difference between UNION and UNION ALL

Why:

Common SQL theory question.

29) What is a view?

Why:

Tests abstraction and reusable queries.

30) Design schema for e-commerce order system

Why:

This is the most important SDE-style question.

They check:

- schema thinking
- relationships
- normalization
- scalability

Project Questions.

Interviewers may ask you to open your GitHub/codebase live, explain the complete flow, justify the syntax used, and write/modify SQL or MongoDB queries based on your project

1) Explain your project in 2 minutes

This is almost always the first question.

What they check: communication + clarity + confidence

You must explain:

- problem statement
- tech stack
- key features

- your contribution
 - outcome
-

2) What problem does your project solve?

Interviewers want to know the **real-world use case**.

Example:

“It helps recruiters screen resumes automatically using AI.”

3) What was your exact contribution?

For freshers, this is one of the most important questions.

They want to know:

- what you built personally
 - whether project is really yours
 - depth of ownership
-

4) Why did you choose this tech stack?

Example:

- Why React?
- Why Spring Boot?
- Why MySQL?
- Why MongoDB?

Prepare **why this and why not alternatives**.

5) Explain end-to-end flow of your project

Example flow:

User → Frontend → API → Backend → Database → Response

6) Explain the architecture of your project

They may ask:

“Can you draw the architecture?”

Prepare:

- modules
 - request flow
 - database layer
 - business layer
 - authentication layer
-

7) Explain your database schema (vvimp)

Very high probability.

Prepare:

- tables
 - primary keys
 - foreign keys
 - relationships
 - normalization
-

8) Why did you use this database design?

This checks design thinking.

Example:

Why separate users, orders, and products tables?

9) What was the hardest challenge in your project?

This question checks **problem-solving**.

Use STAR format:

- Situation
- Task

- Action
 - Result
-

10) How did you optimize performance?

Examples:

- indexing
 - pagination
 - caching
 - lazy loading
 - reducing API calls
-

11) What happens if two users update the same data at the same time?

This checks concurrency thinking.

Possible topics:

- transactions
 - locks
 - optimistic locking
 - race conditions
-

12) How did you handle API failures or exceptions?

Important backend question.

Prepare:

- try/catch
 - global exception handler
 - fallback response
 - retry logic
-

13) Which APIs did you build? Explain one in detail

Prepare one API deeply:

- endpoint
 - request
 - response
 - validation
 - DB call
 - error handling
-

14) How did you secure your application?

Important question.

Possible topics:

- JWT authentication
 - password hashing
 - SQL injection prevention
 - authorization
-

15) Which data structures / algorithms did you use in your project?

Interviewers often connect project with DSA.

Example:

- HashMap for caching
 - priority queue
 - graph / BFS
 - trie for search
-

16) What is the time complexity of your main feature?

Example:

search feature complexity

sorting complexity

recommendation logic complexity

18) What bugs/issues did you face and how did you solve them?

This checks debugging skills.

Prepare 2–3 real examples.

19) If given more time, what improvements would you make?

This checks product thinking.

Examples:

- add caching
 - improve UI
 - better search
 - optimize DB
 - add analytics
-

20) What did you learn from this project?

Usually asked at the end.

Prepare:

- technical learnings
- teamwork
- debugging
- scalability mindset